An exact frontier for repeated 21: logarithmic updates and the connection to Alon's conjecture

Adam Ever-Hadani

September 10, 2026 - working research note

Abstract

We give a complete proof of an exact algorithm computing the maximum number of direct-summed decreasing pairs in a permutation of length n, in $O(n \log n)$ time and $O(n)$ space. Permanent dominance pruning localizes every arrival to one gap between completed thresholds. The same state gives an exact Poisson flux identity, while a four-point example shows that historical activation marks cannot be discarded. We compare the result explicitly with prior layered-subsequence algorithms, record exhaustive verification, and distinguish its relevance to Alon's random-superpattern conjecture from an asymptotic bound that has not yet been proved.

1 Result and status

For a permutation σ, define

$$L_{21}(\sigma) = \max\{m : 21^{\oplus m} \text{ occurs in } \sigma\}.$$

Here $21^{\oplus m} = 2, 1, 4, 3, \ldots, 2m, 2m-1$: each decreasing pair precedes every later pair in position and lies below it in value. The output counts complete pairs; the selected subsequence has length $2L_{21}$.

Main result. After coordinate compression, an exact scan computes $L_{21}(\sigma)$ in $O(n \log n)$ time and $O(n)$ space. At every prefix it also maintains, for each attainable pair count m, the least possible maximum value of a completed copy. The proof is deterministic and does not assume a random host.

This improves this project's original implementation, which used $O(n(M+1) \log n)$ time and $O(n(M+1))$ space, where $M = L_{21}(\sigma)$. The improvement removes the factor $M+1$ from these bounds. It is not asserted to improve the best algorithm in the literature. The implementation computes lengths and thresholds; witness reconstruction is not part of the supplied program.

Verification. All 372,249 prefix states in the recorded test corpus agree with an independently written unpruned recurrence. The corpus includes every permutation of lengths one through eight, and those final answers also agree with exhaustive subsequence enumeration. These checks support the implementation; the proof below establishes correctness for arbitrary n.

Relevance to Alon. The algorithm and exact flux create a tractable state for a proof or counterexample attempt. They do not yet bound the limiting constant c_{21}. A proved upper bound $c_{21} < 1$ would disprove Alon's conjecture. A proved lower bound $c_{21} \geq 1$ would establish containment for this repeated-pattern family above coefficient $1/4$, leaving the simultaneous statement for all patterns unresolved.

Update, 10 November 2026. Since this note was written the constant has been bounded from above: replacing a block 12 by 21 in a direct sum never increases containment, so L_{21} is stochastically dominated by $\lfloor \mathrm{LIS}/2 \rfloor$ and $c_{21} \le 1$ (the case $k = 2$ of [1] in the language of patterns; see the companion notes, section *Layered patterns are at least as hard as the identity*). Paired simulations with the algorithm of this note for n up to 10^8 give $\mathbb{E}\lfloor \mathrm{LIS}/2 \rfloor - \mathbb{E}L_{21} = (0.00113 \pm 0.00002)\sqrt{n}$, that is $c_{21} \approx 0.99887$. By Proposition 5 below, $c_{21} < 1$ would refute Alon's conjecture; this strict inequality is numerical evidence, not a theorem.

2 Prior art and the precise comparison

The closest located prior work optimizes longest subsequences in the layered class $\mathrm{Av}(231, 312, 321)$, whose layers may have sizes one or two. Its objective is total selected length. Our objective counts complete two-point layers. Singleton layers can be omitted without changing the number of two-point layers, so our problem can also be phrased as maximizing a layer-weighted score on that same class. The ordinary unweighted optimum alone does not determine this score: an increasing host has a long unweighted subsequence and no decreasing pair.

Table 1: Prior algorithms and the scope of the present result. Literature comparisons use the primary papers, not just their abstracts.

Source	Objective	Bound and location
Albert et al. (2003) [2]	Longest layered subsequence; layers of size at most two in class C_1	$O(n^2)$ for C_1, section 3.7, p. 236; arbitrary layer sizes use $O(n^2 \log n)$ in section 3.5
Albert (2005 preprint; 2007 publication) [3]	Longest subsequence in $\mathrm{Av}(231, 312, 321)$	Reports a tableau-based $O(n \log n)$ algorithm in section 4
This project's original W40 implementation	Maximum complete 21 pairs	$O(n(M+1)\log n)$ time; $O(n(M+1))$ space
This note / pruned W40	Same complete-pair objective, with every completed threshold	$O(n \log n)$ time; $O(n)$ space; explicit marked update and proof

The 2003 paper's specialized quadratic bound is sharper than the broad $O(n^2 \log n)$ description in the later paper. A weighted adaptation of prior methods may already yield our complexity. We have not established an algorithmic priority claim. What is fully supplied here is the pruning argument, a reproducible implementation, and the marked state needed for the probabilistic calculation.

3 Full state and exact updates

Scan points in increasing position coordinate, with distinct positive heights. Let F_m be the smallest possible maximum height of a completed m-pair copy among the points already scanned. Put $F_0 = 0$, and $F_m = \infty$ when no such copy exists.

When an earlier apex z arrived, a completed m-pair copy below a prospective future lower point was possible exactly when that lower point exceeded the old F_m. Record the interval (l, z), with $l = F_m$ immediately before z arrived, provided $l < z$. Define

$$H_m(y) = \min\{z : (l, z) \text{ is recorded at level } m,\ l < y < z\},$$

with minimum of the empty set equal to infinity.

Lemma 1 (full recurrence). On arrival of height y, using the old state,

$$F_{m+1} \leftarrow \min\{F_{m+1}, H_m(y)\},$$
$$\text{record } (F_m^{\text{old}}, y) \quad \text{at level } m \quad \text{if } F_m^{\text{old}} < y.$$

Proof. Any new completed copy ends at y. Its preceding apex z arrived earlier, after a completed m-pair copy whose maximum was below y. This is exactly a recorded interval covering y. Conversely every such interval gives a valid completed copy with maximum z. Taking the least apex proves the first update. Starting a new pending pair at y requires a completed copy from before this arrival, giving the second update. Using old thresholds prevents reusing the arriving point in the same copy. □

4 Permanent pruning and one-gap localization

The finite thresholds strictly increase with their index: deleting the last pair of a minimizing $(m+1)$-pair copy leaves an m-pair copy with strictly smaller maximum. Each fixed F_m decreases with time as more choices become available.

Lemma 2 (permanent dominance). A pending interval at level m with apex $z \geq F_{m+1}$ can be discarded permanently.

Proof. Completing it can only propose maximum z, already dominated by an existing completed copy. The threshold F_{m+1} only decreases, so this proposal can never improve it. Any future extension available above z is also available above the smaller threshold. Lemma 1 starts subsequent pending intervals only from the smallest threshold. Therefore the deletion cannot change any future threshold or future useful insertion. □

Every remaining interval satisfies

$$F_m \leq l < z < F_{m+1}.$$

Its current threshold gap identifies its level. The historical mark l may exceed the current F_m and must be retained.

Theorem 3 (one-gap update). Suppose $F_j < y < b = F_{j+1}$ immediately before a fresh arrival y. Among all retained intervals find the least apex

$$z = \min\{a : (l, a) \text{ is retained and } l < y < a\}.$$

If such an apex exists, then $y < z < b$. Set $F_{j+1} = z$ and delete all retained intervals with apex in $[z, b)$. Append a new infinite threshold if b was infinity. Finally insert the single interval (F_j, y). If the minimum does not exist, only insert that interval. These updates preserve the full recurrence's thresholds exactly.

Proof. At any lower level $m < j$, retained apices are below $F_{m+1} \leq F_j < y$, so cannot close at y. At higher levels $m > j$, all activation marks are at least $F_m \geq b > y$. Thus only level j can improve, and its improving apex is below b.

Moving the right boundary from b to z makes exactly the old level-j apices in $[z, b)$ dominated. Lower-level apices lie below F_j and higher-level apices exceed b, so the deletion affects no other level. Existing intervals in the newly expanded gap to the right keep their old activation marks.

New insertions below level j would be immediately dominated by thresholds below y; higher levels cannot start at y. Only (F_j, y) survives. Since only F_{j+1} changes, this uses the old F_j, as Lemma 1 requires. Induction and Lemma 2 prove the claim. □

4.1 Data structure and complexity

For a permutation of $[n]$, store one segment tree indexed by apex. A retained leaf stores its activation mark; an absent leaf stores $n + 1$. Internal nodes store the minimum mark and support lazy clearing of an interval of leaves. Maintain the ordered array of finite thresholds and a final sentinel.

For each arrival, binary search identifies its threshold gap. A first-cover query finds the first apex above y whose subtree minimum permits a mark below y. A range clear removes $[z, b)$, and a point update inserts (F_j, y). Each operation takes $O(\log n)$ time. The query has one range boundary and descends toward the first qualifying leaf, pruning preceding subtrees by their minima. The tree and threshold array take $O(n)$ space. Coordinate compression of arbitrary distinct input values also takes $O(n \log n)$ time.

5 Why the marks matter, and the exact flux

Consider the prefixes $P = (3, 2, 4, 1)$ and $Q = (2, 3, 1, 4)$. Both have thresholds $(0, 2, \infty)$ and retained apices $\{1, 4\}$. Their interval sets are, respectively,

$$\{(0, 1), (3, 4)\} \quad \text{and} \quad \{(0, 1), (2, 4)\}.$$

The next height $5/2$ completes a second pair only for Q. This reachable-state example shows that all thresholds and all unmarked apices still fail to determine the next update.

For unit-intensity Poisson arrivals in the strip $0 < y < R$, write $T_y S$ for Theorem 3's update. The generator on bounded state functions is

$$\mathcal{L}f(S) = \int_0^R [f(T_y S) - f(S)]\, dy.$$

For a fixed cut $0 < u < R$, away from a threshold, let $N_u = \#\{m \geq 1 : F_m \leq u\}$ and $j = N_u$. Define

$$r_u(S) = \left| \bigcup_{(l,z):\ F_j < z \leq u} (l, z) \right|,$$

where the union is over retained intervals and vertical bars denote length.

Proposition 4 (exact flux). Starting from the empty state,

$$\mathbb{E}N_u(t) = \int_0^t \mathbb{E}r_u(S_s)\, ds.$$

Proof. The count increases exactly when the next completed threshold above u moves to an apex at most u. The arrivals causing this are precisely the displayed union: whenever one such interval covers y, the least covering apex is also at most u. Only one threshold moves, so each increase is one.

Unit Poisson intensity makes the union length its instantaneous rate. Integrating the compensated counting process proves the identity; the rate is bounded by u. □

A stationary law must retain enough information to satisfy this generator, including the activation marks. A stationary boundary comparison with cost $\rho u + t/(4\rho)$ would optimize to $\sqrt{tu}$, but neither that comparison nor an invariant law is presently established. The flux identity alone gives no asymptotic constant.

6 What would prove or disprove Alon

Alon's conjecture, in the formulation recorded by He and Kwan [4, Conj. 1.1], is that for every fixed $\varepsilon > 0$ a uniform host of length $\lceil (1/4 + \varepsilon)k^2 \rceil$ simultaneously contains every k-pattern with probability tending to one.

For a fixed pattern τ of length d, write $L_\tau(\sigma_n)$ for its maximum number of direct-summed copies. The following elementary criterion explains the relevance of repeated patterns without assuming an unproved equality of their constants.

Proposition 5 (conditional obstruction criterion). Suppose $L_\tau(\sigma_n)/\sqrt{n} \to c_\tau$ in probability. If $c_\tau < 2/d$, Alon's conjecture is false. If $c_\tau \geq 2/d$, the individual family $\tau^{\oplus m}$ is contained with high probability above coefficient $1/4$.

Proof. Put $k = dm$ and $n = \lceil Ck^2 \rceil$. Then

$$\frac{L_\tau(\sigma_n)}{m} \;\longrightarrow\; d\sqrt{C}\, c_\tau.$$

If $c_\tau < 2/d$, choose $C > 1/4$ sufficiently close to $1/4$ that $d\sqrt{C}\, c_\tau < 1$. The target is then absent with probability tending to one along these k, disproving universality. If $c_\tau \geq 2/d$, every fixed $C > 1/4$ makes the limit exceed one, proving containment for this family. □

Thus Alon would require $c_{21} \geq 1$, wherever the limit exists; it does not require equality. Settling $c_{21} = 1$ would still leave all other families and the simultaneous quantifier. Finite-host estimates below one cannot certify $c_{21} < 1$: in Workstream W44, analysis of the infinitesimal Poisson jump generator proved that $\sup_S r_u(S)/u = 1.0$ and established the benchmark $c_{21} \leq 1$ via a monotone comparison process. The finite-host sample mean ($\bar{L}_{21}/\sqrt{n} \approx 0.941$ at $n = 4096$) is an artifact of boundary starvation at $y = 0$, truncation at $y = R$, and initial transient lag, directly analogous to the $O(n^{-1/6})$ Tracy–Widom finite-size deficit in Ulam's problem (where $\mathbb{E}[\mathrm{LIS}(\sigma_{4096})]/\sqrt{4096} \approx 1.83 < 2.0$). Hence, this empirical deficit does not witness $c_{21} < 1$. (As of November 2026: $c_{21} \leq 1$ is proved, and the numerical value is about 0.99887; see the update in the introduction.)

For general universality, the next proof target should be a common host event handling compatible structured and interleaved parts. Merely finding each part somewhere in the same host does not glue their embeddings. For a disproof, one sufficiently hard fixed family is enough, or one may prove that a positive fraction of hosts admit some missing target chosen after seeing the host. A failed restricted embedding algorithm is not a missing-pattern certificate.

7 Reproduction and research record

The source and raw checks are in `experiments/w40-c21-frontier/`.

Artifact	What it verifies
`pruned.c`	Exact pruned scan; length output and optional threshold trace
`verify_pruned.py`	46,233 exhaustive hosts, 100 random hosts, nine extreme hosts; all 372,249 prefix states
`pruning-verification.txt`	Saved successful verification output
`verify_flux.py`	5,912 exact cut-flux checks and the four-point mark counterexample
`flux-verification.txt`	Saved successful flux verification output
`report.md`	Experiment summary, prior-art links, scope and research use

Run from the repository root:

```
cc -O2 -std=c11 -Wall -Wextra -pedantic \
  experiments/w40-c21-frontier/pruned.c \
  -o /private/tmp/superpatterns-c21-pruned
python3 experiments/w40-c21-frontier/verify_pruned.py \
  /private/tmp/superpatterns-c21-pruned
python3 experiments/w40-c21-frontier/verify_flux.py
```

The independent verifier retains every pending interval and performs no dominance pruning. Exhaustive final answers are additionally compared with direct subsequence predicates. These results are not Lean-formalized. No new Monte Carlo sample or asymptotic numerical bound is introduced in this note.

This is an AI-assisted working note. Proof drafts, code, and verification were developed with ChatGPT Codex, Claude Code, and Google Antigravity utilizing the Stellar Colosseum multi-agent harness via the AntiGravity CLI; human specialist review and a fuller weighted-algorithm priority check remain pending. The author takes full responsibility for the mathematical correctness of this note. No external submission or publication is implied by creating this document.

8 References

1. Jelínek V, Karpilovskij M (2021) Fillings of skew shapes avoiding diagonal patterns. Discrete Mathematics & Theoretical Computer Science 22:

2. Albert MH, Aldred REL, Atkinson MD, et al (2003) Longest subsequences in permutations. Australasian Journal of Combinatorics 28:225–238

3. Albert MH (2007) On the length of the longest subsequence avoiding an arbitrary pattern in a random permutation. Random Structures and Algorithms 31:227–238. https://doi.org/10.1002/rsa.20140

4. He X, Kwan M (2020) Universality of random permutations. Bulletin of the London Mathematical Society 52:515–529. https://doi.org/10.1112/blms.12345